On Problems of Implicit Context Compression for Software Engineering Agents

Kirill Gelvan [1,2], Igor Slinko [1], Felix Steinbauer [2], Egor Bogomolov [1], Florian Kofler [2], Yaroslav Zharov [1]

[1]JetBrains Research • [2]Technical University of Munich, Germany • kirill.gelvan@jetbrains.com

LLM-based Software Engineering agents face a critical bottleneck: context length limitations cause failures on complex, long-horizon tasks. One promising solution is to encode context as continuous embeddings rather than discrete tokens, enabling denser information storage. We apply the recently proposed In-Context Autoencoder for this purpose. While the method performs well on single-shot common-knowledge and code-understanding tasks, our experiments demonstrate that it fails on multi-step agentic coding tasks. In this paper, we explore this phenomenon and discuss possible factors contributing to this failure.

Date: April 2026

1 Introduction

Software Engineering (SE) is increasingly automated by Large Language Model (LLM)-based agents that write and fix code (J. Yang, Jimenez, et al., 2024; Jimenez et al., 2023). These agents interact with a development environment over multiple turns to perform complex tasks, such as feature implementation or bug fixing. At each step, the agent takes observations from the environment (e.g., output of a previous command, or input from the user), prompts an LLM to reason about the next action given the history of previous actions and observations, and then executes this action. Different LLM-based agents vary in architectures, but the described framework, called ReAct (S. Yao et al., 2022), is one of the bases on which many are built.

However, the effectiveness of such agents is limited by the amount of information they can process (Shi et al., 2025). This limitation comes from the fixed context window of the underlying models. As agents accumulate observations from tools (e.g., file contents, error logs, and command outputs), they quickly exhaust this window (Kang et al., 2025; Labate et al., 2025). While modern hardware and optimizations allow processing millions of tokens, models fail to function beyond their training context length.

This hard limit leads to a halt in operation or a steep drop in quality when ignored. Additionally, as the context grows, models suffer from the "needle in a haystack" phenomenon, where the accuracy degrades as relevant information becomes buried in long sequences (N. F. Liu et al., 2024). As Badertdinov et al., 2025 show, a single SWE-bench-like issue requires consuming over a million tokens on average.

To address this bottleneck, various solutions have been proposed. Some efforts focus on training models with massive context windows (Comanici et al., 2025) or modifying architectures with mechanisms like Infini-attention (Munkhdalai, Faruqui, and Gopal, 2024). Others employ mechanisms to mitigate Rotary Positional Embeddings (RoPE) (Su et al., 2021) extrapolation issues, such as NoPE (Kazemnejad et al., 2023) or DroPE (Gelberg et al., 2025), or strategies that explicitly drop or compress earlier parts of the interaction trajectory to save space (Pan et al., 2024; Wang et al., 2024).

Implicit context compression represents a particular compression approach where text is encoded into continuous embeddings rather than discrete tokens (Dai et al., 2025). This approach operates under the assumption that dense real-valued vector representations can hold more information than discrete tokens (Morris et al., 2023). Consequently, encoders are trained to reduce the number of required "attention

slots" by generating fewer embeddings than there were tokens in the input. Recent works demonstrate that reasoning chains can be effectively compressed into a few continuous embeddings (Hao et al., 2024; Shen et al., 2025; Xu et al., 2025), suggesting the potential for long agentic workflows.

In this work, we investigate implicit context compression for SWE agents using the In-Context Autoencoder (ICAE) approach (Ge et al., 2023) (detailed in Section 2). This method proposes an encoder-decoder architecture to compress the input context into continuous embeddings. We adapt ICAE for SE agent tasks by fine-tuning on multi-step trajectories from the SWE-Smith dataset (J. Yang, Lieret, et al., 2025) and evaluate its performance on the SWE-bench Verified benchmark (Chowdhury et al., 2024). We observe a surprising lack of generalization from single-step tasks to multi-step trajectories, investigate possible reasons with additional experiments in Section 3, and propose two potential explanations in Section 4.

2 Method

This section details the adaptation of implicit context compression to agentic SE tasks, based on the In-Context Autoencoder (Ge et al., 2023). We outline the original ICAE architecture, its standard pretraining and fine-tuning procedures, and our modifications to them. We then present our modifications for the agentic domain.

Architecture. The ICAE architecture consists of two decoder-only transformers: a trainable encoder and a frozen decoder, both initialized from the same pretrained foundation model. The encoder is trained with Low-Rank Adaptation (LoRA) (Hu et al., 2022) to compress long contexts into a fixed sequence of continuous embeddings, called *memory tokens*. The decoder attends to these memory tokens instead of the original text to generate output. Unlike other approaches (Bulatov, Kuratov, and Burtsev, 2022; Jaegle et al., 2021), the decoder remains frozen, preventing catastrophic forgetting (McCloskey and Cohen, 1989). We replace the Llama-2-7B base model (Touvron et al., 2023) with Qwen3-8B (A. Yang et al., 2025), as the latter is reported to perform better on agentic tasks.

Pretraining. The goal of pretraining is to initialize the compression capability using a massive text corpus. Ge et al. (2023) originally used The Pile (L. Gao et al., 2020). Training employs a 50/50 mix of autoencoding (reconstructing the input text verbatim) and language modeling (continuation of the compressed text) objectives. To make the decoder aware of the task, the compressed memory tokens are appended with a special task-signaling token. For both objectives, the decoder generates the output based on the compressed context, but *only the encoder weights* are optimized. This forces the encoder to learn representations that preserve semantic information in a format the frozen decoder can understand. Due to the DMCA takedown[1] of The Pile dataset, we instead use SlimPajama-6B (Weber et al., 2024) for pretraining.

Fine-tuning. The next stage fine-tunes the encoder for specific downstream tasks (originally for Question Answering). The model receives a long context, which the encoder compresses into memory tokens. The user's question is kept in its original discrete token form and appended to the memory tokens. The frozen decoder then attends to both the memory tokens representing the document and the discrete tokens of the question to generate the answer. Gradients from the answer generation are backpropagated to update the encoder, optimizing the compressed representation for the task.

Agentic Fine-Tuning. We adapt ICAE for agentic workflows by modifying the task structure. For fine-tuning on agentic trajectories, we compress only observations longer than 256 tokens and compute backpropagation only for the latest compressed observation. Due to memory constraints, all previous observations are kept cached as memory tokens until the trajectory is finished. Short observations, actions, and the system prompt are preserved in their original discrete token form. The model is trained to predict the next action (tool call) based on a history containing compressed observations. Following Zhao et al. (2024), we employ position ID manipulation to minimize the effective distance between memory tokens and the current prompt, facilitating better attention mechanisms in RoPE-based models. Appendix D further illustrates the training and inference process.

[1] https://en.wikipedia.org/wiki/The_Pile_(dataset)#Training_on_copyrighted_works_or_derivatives

3 Experiments

We evaluate our modified version of ICAE across three distinct scenarios for context compression. The first one tests the model's ability to retrieve explicit facts from compressed natural language using **SQuAD** (Rajpurkar et al., 2016). The second assesses whether the compression preserves the high-fidelity details required for code comprehension using **RepoQA** (J. Liu et al., 2024). The third evaluates the model's capacity to plan and execute multi-step SE tasks using **SWE-bench Verified** (Chowdhury et al., 2024).

To assess performance across these scenarios, we compare three configurations: (i) **Base** – Qwen3-8B (A. Yang et al., 2025) without fine-tuning or compression, (ii) **SFT** – Qwen3-8B fine-tuned on the respective task without compression, and (iii) **ICAE** – our modification, with the encoder pretrained and then fine-tuned as described in Section 2. We disable thinking for all experiments to ensure comparable results. All quantitative results are presented in Table 1 and discussed further in this Section. For the sake of space, we share all training hyper-parameters in Appendix A and detailed experiment statistics in Appendix B.

Model	Compress. Rate	SQuAD		RepoQA		SWE-bench Verified	
		BLEU	EM	BLEU	Pass@0.8	BLEU_{ref}	Resolved Issues
Base	1×	0.67	0.54	0.81	0.65	0.48	19
SFT	1×	**0.75** (+0.08)	**0.70** (+0.16)	**0.90** (+0.09)	**0.85** (+0.20)	**0.55** (+0.07)	**86** (+67)
ICAE	4×	0.73 (+0.06)	0.67 (+0.13)	0.87 (+0.06)	0.69 (+0.04)	0.51 (+0.03)	7 **(-12)**

Table 1. Performance report. Best results in **bold**, second best underlined. Performance differences with Base model are reported in parentheses. SFT uses LoRA for SQuAD and RepoQA, but full fine-tuning for SWE-bench Verified.

SQuAD. In SQuAD, each sample consists of three parts: a question, a context paragraph, and an answer. Given the question and the context paragraph, the model is expected to provide the correct answer. We fine-tune the encoder on the training set to compress the context paragraph. The fine-tuning target is for the decoder to take as input the uncompressed question and compressed context and predict the answer. We evaluate performance on the standard validation set, reporting Bilingual Evaluation Understudy (BLEU) scores (Papineni et al., 2002) and the Exact Match (EM). We observe that ICAE significantly improves over the baseline ($p < 0.001$), while underperforming the SFT checkpoint. This confirms that compression aids in extracting knowledge from general text but has a minor penalty compared to full-context access.

RepoQA. We utilize the "Searching Needle Function" task from the RepoQA dataset to test code retrieval. In this task, the model is required to retrieve a specific function from the whole code repository given in the context, based on a short natural language description. The fine-tuning target is to take as input the uncompressed instructions and function description and compressed code to predict the target function. We use a context length of 8192 tokens to verify high-fidelity retrieval. We follow the authors and use BLEU and Pass@0.8 metrics averaged across all five programming languages in the benchmark. Pass@0.8 measures the percentage of generations that achieve a BLEU score of at least 0.8 with the ground truth, and is calculated after extracting the code block from the generated string and removing comments. Across 5 runs, we find no statistically significant difference between ICAE and Base, confirming that compression maintains retrieval capabilities, whereas SFT yields substantially larger gains on the downstream metric.

SWE-bench Verified. We use the whole dataset of 500 SE tasks as the evaluation set. This dataset consists of 12 repositories, issue texts for each repository, and a set of tests for each issue. A successful task resolution is registered if all corresponding tests pass after the proposed code patch is applied. The fine-tuning target is for the decoder to predict the next action given the series of uncompressed actions and compressed observations. We use expert successfully resolved agentic trajectories from the SWE-smith dataset (J. Yang, Lieret, et al., 2025) for fine-tuning, reserving 300 trajectories as a test set. Due to the complexity of the task, the SFT uses full fine-tuning instead of LoRA. However, to remain consistent with the original paper, our ICAE continues to use the same training hyperparameters as before. Since this benchmark doesn't provide ground truth actions, as an intermediate metric of quality, we measure BLEU_{ref}, which stands for BLEU with respect to the expert trajectories from the test subset. As the

downstream metric, we use the number of resolved issues. The BLEU_{ref} metric follows the usual trend — ICAE outperforms the base model and is outperformed by SFT. However, the downstream metric results are drastically different — the ICAE significantly underperforms both the Base model (12 fewer issues solved, $p = 0.013$), and the SFT checkpoint (79 fewer issues solved).

While the method fails to increase the number of resolved issues, it successfully extends the effective context window size. The model was trained with a 4× compression rate, but the effective compression rate, which depends on the dataset characteristics and which parts are selected for compression, is lower. The effective compression is 1.46× on SQuAD, 3.74× on RepoQA, and on 2.0× SWE-bench Verified. We note that the effective compression rate does not correlate with the downstream performance. Practically, on SWE-bench Verified, the method allows for 40% longer trajectories (113 vs. 81 steps on average; see Appendix C) and 10% faster generation time (incl. compression).

4 Discussion

The results show that compression provides a similar or slightly increased downstream performance for single-shot experiments. However, in the multi-step setting, the ICAE underperforms the base model. Since the decoder wasn't trained, we can attribute the problem to the compression mechanism. We identify two hypotheses to explain the failure of implicit compression in the agentic setting: error accumulation and failure to account for long-range dependencies.

Reconstruction Fidelity and Error Accumulation concerns the fidelity of the compressed representation. Through qualitative analysis, we observed examples where URLs or file paths were hallucinated during reconstruction, such as replacing `swe-agent.com/latest/` with `swe-agent.com/agent/latest/`, invalidating the output. In single-step tasks, such errors are isolated to a single output. However, in multi-step agentic workflows, these errors compound.

Information Preservation for Future Steps relates to the limitations of the compression objective during fine-tuning. In our fine-tuning setup, at timestep k, all the observations $1, \ldots, k-1$ are compressed. However, only the last encoder instance (the weights that compressed the observation $k-1$) is optimized. Hence, the training signal flows through only this single step, and the model has no incentive to preserve the information needed for subsequent steps. Nevertheless, since each step involves a full pass through two LLMss (encoder and decoder), addressing this would make the training of such architectures computationally very challenging for long sequences.

To confirm or refute these hypotheses, we propose future work to experiment with a more granular selection of steps for compression. In this work, we selected observations to compress based on their length; however, further investigation of failure patterns may help to pinpoint the problem. For example, compressing only the last k steps may test the **information preservation** hypothesis, and randomly compressing k% of steps may estimate the compression losses regarding **reconstruction fidelity**.

5 Conclusion

In this paper we evaluated implicit context compression for LLM-based SWE agents. While ICAE successfully improves efficiency (40% longer trajectories, 10% faster generation) and outperforms the baseline on single-shot tasks (general and coding), it fails in the multi-step agentic SE setup. Specifically, we observe a significant performance degradation on SWE-bench Verified compared to the baseline model.

Our analysis suggests two possible contributing factors: insufficient reconstruction fidelity, which compounds errors over time, and a failure to selectively preserve crucial information for future steps due to single-step training objectives. These challenges are non-trivial to resolve and require a more involved modeling and training approach than anticipated (e.g., full-trajectory reinforcement learning). We suggest that future work should focus on identifying the specific challenges in adapting implicit compression to agentic tasks.

LLM Usage Statement

We utilized LLMs for initial drafting and grammar polishing. However, the substantial writing, all experimental design and analysis remain the original contribution of the authors.

Reproducibility Statement

To support reproducibility, we release our code and model checkpoints. The source code for the ICAE implementation, data processing, and evaluation scripts is available at https://github.com/AnonForConference/paper1-code-0126. The model checkpoints, including the pretrained encoder and the agentic fine-tuned model, are hosted at https://huggingface.co/AnonForConference/paper1-model-0126.

References

Badertdinov, Ibragim et al. (2025). "SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents". In: *arXiv preprint arXiv:2505.20411*.

Bulatov, Aydar, Yury Kuratov, and Mikhail Burtsev (2022). "Recurrent memory transformer". In: *Advances in Neural Information Processing Systems* 35, pp. 11079–11091.

Chowdhury, Neil et al. (2024). *Introducing SWE-bench Verified*. OpenAI Blog. URL: https://openai.com/index/introducing-swe-bench-verified/.

Comanici, Gheorghe et al. (2025). "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities". In: *arXiv preprint arXiv:2507.06261*.

Dai, Yuhong et al. (2025). "Pretraining context compressor for large language models with embedding-based memory". In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28715–28732.

Gao, Leo et al. (2020). "The Pile: An 800GB Dataset of Diverse Text for Language Modeling". In: *arXiv preprint arXiv:2101.00027*.

Ge, Tao et al. (2023). "In-context autoencoder for context compression in a large language model". In: *arXiv preprint arXiv:2307.06945*.

Gelberg, Yoav et al. (2025). "Extending the Context of Pretrained LLMs by Dropping Their Positional Embeddings". In: *arXiv preprint arXiv:2512.12167*.

Hao, Shibo et al. (2024). "Training Large Language Models to Reason in a Continuous Latent Space". In: *arXiv preprint arXiv:2412.06769*.

Hu, Edward J et al. (2022). "Lora: Low-rank adaptation of large language models". In: *ICLR*.

Jaegle, Andrew et al. (2021). "Perceiver io: A general architecture for structured inputs & outputs". In: *arXiv preprint arXiv:2107.14795*.

Jimenez, Carlos E et al. (2023). "Swe-bench: Can language models resolve real-world github issues?" In: *arXiv preprint arXiv:2310.06770*.

Kang, Minki et al. (2025). "Acon: Optimizing context compression for long-horizon llm agents". In: *arXiv preprint arXiv:2510.00615*.

Kazemnejad, Amirhossein et al. (2023). "The impact of positional encoding on length generalization in transformers". In: *Advances in Neural Information Processing Systems* 36, pp. 24892–24928.

Labate, Anton Bulle et al. (2025). "Solving Context Window Overflow in AI Agents". In: *arXiv preprint arXiv:2511.22729*.

Liu, Jiawei et al. (2024). "Repoqa: Evaluating long context code understanding". In: *arXiv preprint arXiv:2406.06025*.

Liu, Nelson F et al. (2024). "Lost in the middle: How language models use long contexts". In: *Transactions of the Association for Computational Linguistics* 12, pp. 157–173.

McCloskey, Michael and Neal J Cohen (1989). "Catastrophic interference in connectionist networks: The sequential learning problem". In: *Psychology of learning and motivation*. Vol. 24. Elsevier, pp. 109–165.

Morris, John et al. (2023). "Text embeddings reveal (almost) as much as text". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12448–12460.

Munkhdalai, Tsendsuren, Manaal Faruqui, and Siddharth Gopal (2024). "Leave no context behind: Efficient infinite context transformers with infini-attention". In: *arXiv preprint arXiv:2404.07143* 101. https://arxiv.org/abs/2404.07143.

Pan, Zhuoshi et al. (2024). "Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression". In: *arXiv preprint arXiv:2403.12968*. https://arxiv.org/abs/2403.12968.

Papineni, Kishore et al. (2002). "Bleu: a method for automatic evaluation of machine translation". In: *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. https://aclanthology.org/P02-1040.pdf, pp. 311–318.

Rajpurkar, Pranav et al. (2016). "Squad: 100,000+ questions for machine comprehension of text". In: *arXiv preprint arXiv:1606.05250*.

Shen, Zhenyi et al. (2025). "Codi: Compressing chain-of-thought into continuous space via self-distillation". In: *arXiv preprint arXiv:2502.21074*.

Shi, Yaorui et al. (2025). "Look back to reason forward: Revisitable memory for long-context llm agents". In: *arXiv preprint arXiv:2509.23040*.

Su, Jianlin et al. (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *arXiv preprint arXiv:2104.09864*.

Touvron, Hugo et al. (2023). "Llama 2: Open foundation and fine-tuned chat models". In: *arXiv preprint arXiv:2307.09288*.

Wang, Yan et al. (2024). “Natural is the best: Model-agnostic code simplification for pre-trained large language models”. In: *Proceedings of the ACM on Software Engineering* 1.FSE. https://arxiv.org/abs/2405.11196, pp. 586–608.

Weber, Maurice et al. (2024). “Redpajama: an open dataset for training large language models”. In: *Advances in neural information processing systems* 37, pp. 116462–116492.

Xu, Yige et al. (2025). “Softcot: Soft chain-of-thought for efficient reasoning with llms”. In: *arXiv preprint arXiv:2502.12134*.

Yang, An et al. (2025). “Qwen3 technical report”. In: *arXiv preprint arXiv:2505.09388*.

Yang, John, Carlos E Jimenez, et al. (2024). “Swe-agent: Agent-computer interfaces enable automated software engineering”. In: *Advances in Neural Information Processing Systems* 37, pp. 50528–50652.

Yang, John, Kilian Lieret, et al. (2025). “Swe-smith: Scaling data for software engineering agents”. In: *arXiv preprint arXiv:2504.21798*.

Yao, Shunyu et al. (2022). “React: Synergizing reasoning and acting in language models”. In: *The eleventh international conference on learning representations*.

Zhao, Runsong et al. (2024). “Position IDs Matter: An Enhanced Position Layout for Efficient Context Compression in Large Language Models”. In: *arXiv preprint arXiv:2409.14364*.

A Training Hyperparameters

Table 2 summarizes the training hyperparameters.

Parameter	Pretraining	Fine-tuning
Base Model	Qwen3-8B	Qwen3-8B
Optimizer	AdamW	AdamW
Learning Rate	1×10^{-4}	5×10^{-5}
Batch Size	1	1
Gradient Accumulation	8	1
LoRA Rank	128	128
LoRA α	32	32
Target Modules	q_proj, v_proj	q_proj, v_proj
Warmup Steps	300	300

Table 2. Training hyperparameters for pretraining and fine-tuning stages. We utilize the same hyperparameters across all fine-tuning datasets, varying only the number of training steps: 100,000 for Pretraining, 10,000 for SQuAD, 4,000 for RepoQA, and 150,000 for SWE-bench Verified.

Here we present the agent's scaffolding details. The agent interacts with the environment using one of the three tools:

1. `bash`: standard shell interface for running commands;
2. `submit`: submits the final patch for evaluation; and
3. `str_replace_editor`: stateful file editor supporting view, create, str_replace, insert, and undo_edit operations.

The `str_replace_editor` requires exact line matching for replacements, ensuring deterministic edits. This precise control is essential for making targeted code changes but also means that small reconstruction errors in compressed observations can lead to failed edit attempts.

This interaction protocol is adopted from SWE-smith, where the complete system prompt and tool specifications can be found.

B Detailed Multi-Run Experiment Results

In this section, we list more detailed results of the multiple runs for our experiments and describe the statistics. To confirm statistical significance, we performed a two-sample Welch t-test across 5 independent runs. The results, including the statistical significance are shown in Figure 1

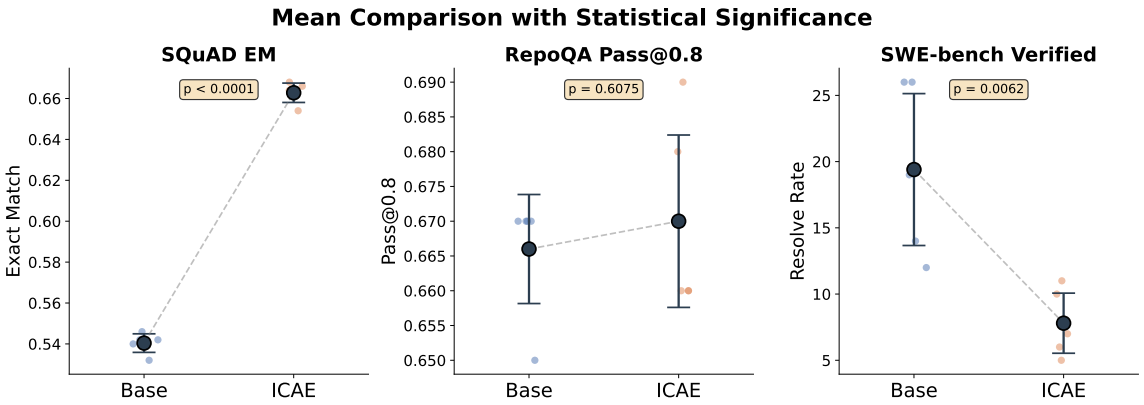


Figure 1. Comparison of Base and ICAE performance across three benchmarks. Each subplot shows mean values with 95% confidence intervals (error bars) and individual run results (scattered points). Statistical significance was assessed using independent samples t-tests. ICAE significantly outperforms Base on SQuAD EM ($p < 0.0001$), shows no significant difference on RepoQA Pass@0.8 ($p = 0.6075$), and performs significantly lower on SWE-bench Verified resolve rate ($p = 0.0062$).

C Trajectory Length Comparison

Figure 2 illustrates the trajectory length distribution. The compressed agent (ICAE) executes significantly more steps before reaching the context limit compared to the uncompressed baseline.

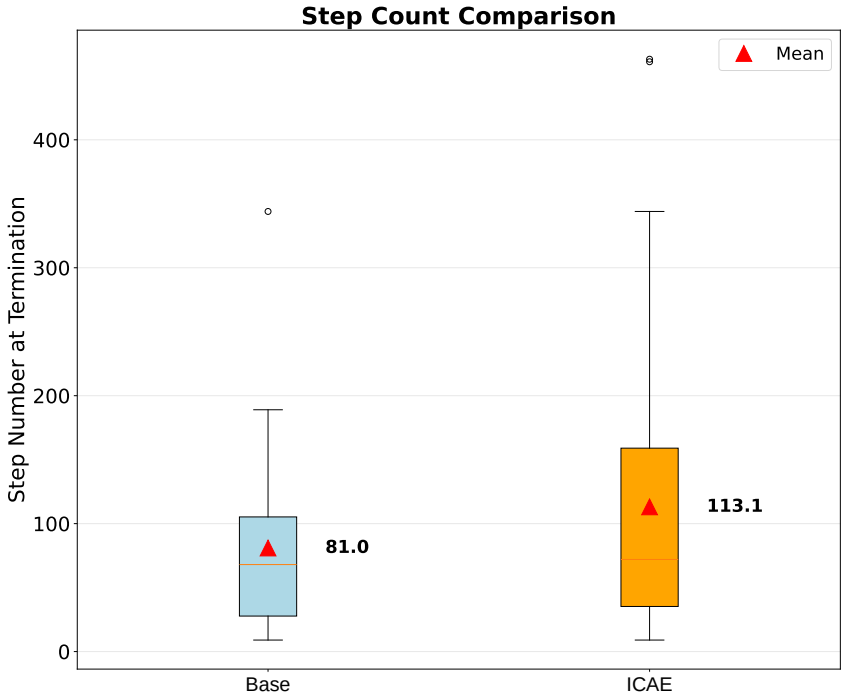
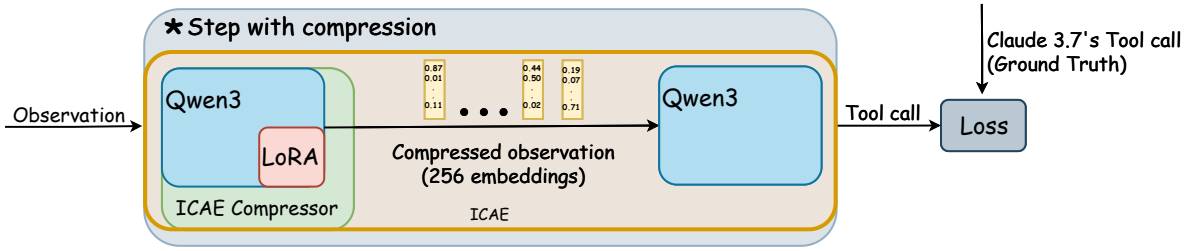


Figure 2. Comparison of trajectory lengths of ICAE and Base model. Comparison is performed at termination (context limit 32k tokens, no step limit). Triangles indicate mean values across models.

D ICAE Training and Inference Details

In this section, we further detail the ICAE training and inference. The following example demonstrates a hand-crafted scenario, closely related to SWE-bench. This scenario was designed to test the model's ability to comprehend and utilize decompressed data from its memory. The environment for this sample was manually simulated by one of the authors. In this particular example, the model is able to run the secret command that has only been observed in the compressed form of memory tokens.

(a) Overview of the agentic trajectory generation process used for fine-tuning data. The step marked with asterisk (Step k) is selected for compression because the previous observation has length of more than 256 tokens.



(b) A single fine-tuning step for the agentic ICAE. Observation from the previous step ($k-1$) is passed through ICAE-encoder, and then the next tool call is generated by ICAE-decoder. The loss is calculated between the ground truth and predicted tool call. The optimization is then applied only to the LoRA weights in the current ICAE-encoder.

Figure 3. Training of ICAE-encoder for the agentic setup. The step selection is illustrated in subfigure (a), and the optimization step is detailed in subfigure (b).

```
User    <uploaded_files>/mnt/shared-fs/swe-agent-distillation</uploaded_files>
          I've uploaded a python code repo. Consider the PR description:
          Your task is to read README.md and run the commands for the setup.
          when i run them i get errors.
          <fixed_part_of_the_first_user_message>*
Agent   $ str_replace_editor --command view --path /mnt/shared-fs/.../README.md
User     <MEMORY_TOKEN> x 512
Agent   $ cat /mnt/shared-fs/swe-agent-distillation/requirements.txt
User    There is no file named requirements.txt
Agent   $ str_replace_editor --command view --path /mnt/shared-fs/swe-agent-distillation
User    You have no access to this directory.
Agent   $ ls -la /mnt/shared-fs/swe-agent-distillation
User    You have no access to this directory. Call a command from README.md to install.
Agent   $ str_replace_editor --command view --path /mnt/shared-fs/.../README.md
User     <MEMORY_TOKEN> x 512
Agent
 $ pip install -r /mnt/shared-fs/swe-agent-distillation/.my_secret_requirements.txt
```

Figure 4. *`<fixed_part_of_the_first_user_message>` follows the SWE-smith system prompt format and is omitted for brevity.